

目录

目录

一. 系统设计	2
二. 模块设计	3
输入输出规划描述:	3
1. TOP.v 顶层模块	3
2. KEY_JITTER.v 按键消抖模块	6
3. CTRL.v 主控模块	7
4. 显示模块:	14
5. 三色 LED 显示模块:	17
6. 蓝牙模块:	20
硬件接口对应:	27
三. 实验结果及结论	28
1. 仿真方法和仿真文件	29
2. 实物操作如下:	37
四. 设计特色, 及设计过程中遇到的问题描述和解决方法	45
1. 04023301 何君个人总结	45
2. 04023426 唐熙文个人总结:	48
五. 人员任务分配情况	48
六. 其他	49

一. 系统设计

功能描述：用于模仿密码锁的工作过程。完成密码锁的核心控制功能。

功能要求及验收要点：设计一个密码锁，平时处于**等待状态**。

- (1) 管理员可以通过设置（专用按键）更改密码。
- (2) 如果没有预置密码，密码缺省为“0000”。
- (3) 用户如果需要开锁，拨动**相应的开关**进入**输入密码状态**，输入**4位**密码，按下确定键后，若密码正确，锁打开，若密码错误，将提示密码错误，要求重新输入，**三次输入都错误，将发出报警信号**。
- (4) 报警后，只有**管理员**作相应的处理（**专用按键**）才能停止报警。
- (5) 用户输入密码时，在按下确定键之前，可以通过按**退格键修正**，每按一次退格键消除一位密码。
- (6) 正确开锁后，用户处理完毕后，按下确定键，系统回到等待状态。
- (7) 系统操作过程中，**只要密码锁没有打开**，如果10秒没有对系统操作，**系统回到等待状态**。
- (8) 系统操作过程中，如果密码锁已经打开，如果**20秒没有对系统操作**，系统**自动上锁**，回到**等待状态**。

提示：

- (1) 密码正确，锁打开时，可以使用开关上方的LED灯配合显示效果，比如**LED全亮**等。密码错误，提示信号也可以使用LED进行显示。报警信号也可以使用LED进行显示，比如**不停的闪烁**等。
- (2) 数码管要充分使用，用**以显示用户输入的数字**等。

①简要分析

设计密码锁控制器。模仿密码锁的工作过程。完成密码锁的核心控制功能。

该系统设计旨在按照时序逻辑，接收用户的输入信号，验证输入序列是否正确，并由此改变密码锁的开关状态，控制密码锁，输出指示信息。

②系统顶层设计

主控部分主要负责题目功能的实现，设计题目设计要求的完整的状态转换。为完成所有操作，我们设置了七种状态：等待状态、输入状态、开锁状态、错误状态、报警状态、管理员状态和比较状态。在该模块中，我们读取对应开关和按键的状态，根据状态转换图完成具体操作。

③输入输出（I/O）规划描述

(1) 输出模块

- a) 在输入状态下，按下确认键，系统开始将输入的数字与密码进行比较，从而跳转到下一个状态；
- b) 开锁状态下按下确认键，系统回到等待状态；
- c) 在管理员状态下按下确认键，密码修改完毕。
- d) 管理员按键：下方按键。
- e) 在报警状态下按下该键，报警解除，回到等待状态；
- f) 在其他状态下按下该键，开始修改密码；

(2) 显示模块：

显示模块主要负责段码的显示。根据状态的不同，段码会有不同的显示。

- a. 等待状态下，左侧四位显示；

- b. 输入密码状态下，根据拨动开关输入的数字，数码管左侧四位依次显示，未输入的位数仍显示；同时在输入状态下，最右侧一位数码管显示数字，代表输错的次数；
- c. 若密码正确，左起五位数码管显示“HELLO”；
- d. 若密码错误 1 次或者 2 次，左起五位数码显示“ERROR”；
- e. 若密码错误 3 次，左侧四位数码管显示“EEEE”，并且蜂鸣器进行周期为 1s 的鸣叫；
- f. 管理员状态下，数码管显示_ _ _ _，并等待拨码开关的输入从而依次进行显示。

二. 模块设计

主控部分主要负责题目功能的实现，设计题目设计要求的完整的状态转换。为完成所有操作，我们设置了七种状态：等待状态、输入状态、开锁状态、错误状态、报警状态、管理员状态和比较状态。在该模块中，我们读取对应开关和按键的状态，根据状态转换图完成具体操作。

输入输出规划描述：

1. TOP. v 顶层模块

顶层模块主要负责定义程序所需的输入和输出信号，同时也定义了所需的内部信号，将各个子模块相连，从而实现模块化设计。

```
//=====
// 物理按键消抖模块实例化
//=====
KEY_JITTER i0_KEY_JITTER(
    .CLK(CLK),
    .key_in(START_IN),
    .key_posedge(START_PHYSICAL)
);

KEY_JITTER i1_KEY_JITTER(
    .CLK(CLK),
    .key_in(BACKSPACE_IN),
    .key_posedge(BACKSPACE_PHYSICAL)
);

KEY_JITTER i2_KEY_JITTER(
    .CLK(CLK),
    .key_in(OK_IN),
    .key_posedge(OK_PHYSICAL)
);
```

```
KEY_JITTER i3_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(MANAGER_IN),  
    .key_posedge(MANAGER_PHYSICAL)  
);
```

```
KEY_JITTER S0_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW0_IN),  
    .key_posedge(SW0_PHYSICAL)  
);
```

```
KEY_JITTER S1_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW1_IN),  
    .key_posedge(SW1_PHYSICAL)  
);
```

```
KEY_JITTER S2_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW2_IN),  
    .key_posedge(SW2_PHYSICAL)  
);
```

```
KEY_JITTER S3_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW3_IN),  
    .key_posedge(SW3_PHYSICAL)  
);
```

```
KEY_JITTER S4_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW4_IN),  
    .key_posedge(SW4_PHYSICAL)  
);
```

```
KEY_JITTER S5_KEY_JITTER(  
    .CLK(CLK),  
    .key_in(SW5_IN),  
    .key_posedge(SW5_PHYSICAL)  
);
```

```
KEY_JITTER S6_KEY_JITTER(  
    .CLK(CLK),
```

```

        .key_in(SW6_IN),
        .key_posedge(SW6_PHYSICAL)
    );

    KEY_JITTER S7_KEY_JITTER(
        .CLK(CLK),
        .key_in(SW7_IN),
        .key_posedge(SW7_PHYSICAL)
    );

    KEY_JITTER S8_KEY_JITTER(
        .CLK(CLK),
        .key_in(SW8_IN),
        .key_posedge(SW8_PHYSICAL)
    );

    KEY_JITTER S9_KEY_JITTER(
        .CLK(CLK),
        .key_in(SW9_IN),
        .key_posedge(SW9_PHYSICAL)
    );

    //=====
    // 蓝牙接口模块实例化
    //=====
    BLUETOOTH_INTERFACE bt_interface (
        .CLK(CLK),
        .RESET(RESET),
        .RX(BT_RX),
        .TX(BT_TX),

        // 蓝牙输出信号 - 连接到组合逻辑
        .BACKSPACE_BT(BACKSPACE_BT),
        .OK_BT(OK_BT),
        .MANAGER_BT(MANAGER_BT),
        .START_BT(START_BT),
        .SW0_BT(SW0_BT), .SW1_BT(SW1_BT), .SW2_BT(SW2_BT), .SW3_BT(SW3_BT), .SW4_BT(
SW4_BT),
        .SW5_BT(SW5_BT), .SW6_BT(SW6_BT), .SW7_BT(SW7_BT), .SW8_BT(SW8_BT), .SW9_BT(
SW9_BT),

        // 状态反馈 - 从 CTRL 模块获取当前状态
        .CURRENT_STATE(STATE),
        .ERROR_TIMES(error_count),    // 如果 CTRL 有错误计数则连接, 否则保持 3'b0
    );

```

```

        .LED_STATUS(LED)
    );

//=====
// 主控制模块 - 使用组合后的信号作为输入
//=====
CTRL i_CTRL(
    .CLK(CLK),
    .RESET(RESET),

    // 使用组合后的信号（物理输入 OR 蓝牙输入）
    .BACKSPACE(BACKSPACE),
    .OK(OK),
    .MANAGER(MANAGER),
    .START(START),
    .SW0(SW0), .SW1(SW1), .SW2(SW2), .SW3(SW3), .SW4(SW4),
    .SW5(SW5), .SW6(SW6), .SW7(SW7), .SW8(SW8), .SW9(SW9),

    // 输出信号
    .STATE(STATE),
    .LED(LED),
    .AN(AN),
    .SEG(SEG),
    .RGB1_RED(RGB1_RED), .RGB1_GREEN(RGB1_GREEN), .RGB1_BLUE(RGB1_BLUE),
    .RGB2_RED(RGB2_RED), .RGB2_GREEN(RGB2_GREEN), .RGB2_BLUE(RGB2_BLUE),

    // 如果 CTRL 模块有错误计数输出，可以在这里添加
    // .ERROR_COUNT(error_count)
    .BUZZER(BUZZER) // 连接蜂鸣器输出
);

```

2. KEY_JITTER. v 按键消抖模块

由于按键可能产生抖动，导致按下按键后出现系统混乱的结果；所以。为了保证系统的稳定，必须对按键进行消抖处理；

消抖的原理如下：我们将按键在相邻两个时钟周期的状态赋给按键状态寄存器，并设置一个时钟的计数器。取持续的 10ms 时间，如果按键状态有变化，视为按键抖动，不记录键值，计数器清 0；如果在 10ms 内按键状态没有发生变化，视为按键被按下，将按键值赋给按键值寄存器，并在一个时钟周期后将其赋给延迟一个周期的按键值寄存器，并作相应的逻辑计算，最终能够实现输出一个时钟周期的高电平信号，检测到上升沿，完成一个按键被按下的操作。

以 SW0 为例的按键消抖模块如下：

```
module KEY_JITTER(  
    input CLK,  
    input key_in, //按键输入信号  
    output key_posedge //按键上升沿检测信号（输出）  
);  
//内部信号  
reg [1:0] key_in_r; //按键输入的寄存器  
wire r; //按键状态寄存器  
reg [19:0] count; //计数器  
reg key_value_r = 0; //按键值的寄存器  
reg key_value_rd = 0; //按键值的寄存器（延时一个时钟周期）  
//将当前的按键输入保存到 key_in 寄存器中  
always @(posedge CLK)  
    key_in_r <= { key_in_r[0], key_in };  
//检测有输入有没有变化  
assign r = key_in_r[0]^key_in_r[1];  
  
always @(posedge CLK)  
    if(r == 1'b1)  
        count <= 20'h0; //连续检测到按键输入有变化，计数器清 0  
    else  
        count <= count + 1;  
//count 达最大值时，将当前 key_in_r[0] 赋给 key_value_r  
always @(posedge CLK)  
    if(count == 1000000)  
        key_value_r <= key_in_r[0];  
//上升沿时赋值，实现一个时钟周期的延迟  
always @(posedge CLK)  
    key_value_rd <= key_value_r;  
//将 key_posedge 赋值为按键上升沿检测信号，按键在上升沿时为逻辑真  
assign key_posedge = key_value_r & ~key_value_rd;  
  
endmodule
```

3. CTRL.v 主控模块

主控部分主要负责题目功能的实现，设计题目设计要求的完整的状态转换。为完成所有操作，我们设置了七种状态：等待状态、输入状态、开锁状态、错误状态、报警状态、管理员状态和比较状态。在该模块中，我们读取对应开关和按键的状态，根据状态转换图完成具体操作。

同时为了实现倒计时操作，我们在主控模块中加入了 LED 灯的流水显示工作。在每一次

检测到有操作进行时，根据当前的状态为计数器赋予对应的计数上限，接着对其进行递减操作，实现倒计时。

下面是具体的操作及实现效果

A. 重置键：按下中间按钮，系统重置回到等待状态，且密码被重置为 0000。

```
if(RESET)                                //重置按钮，密码清 0
begin
    Key[0] <= 0;
    Key[1] <= 0;
    Key[2] <= 0;
    Key[3] <= 0;
    Current_State <= WAIT;
    Error_Times <= 0;
end
```

B. 开始键：在等待状态或者错误状态下，按下左侧按钮，进入输入状态，用户可依次输入密码。

```
if(START)                                //按下开始键开始输入密码
begin
    Current_State <= INPUT;
    Count_CLK <= tens;
end

end

if(Current_State == INPUT)                //输入状态下的操作及状态转换
begin
    if(MANAGER)                            //修改密码按键
    begin
        Code_Bit <= 0;
        Code[0] <= 10;
        Code[1] <= 10;
        Code[2] <= 10;
        Code[3] <= 10;
        Current_State <= ADMIN;
        Count_CLK <= tens;
    end

    if(SW0)                                //检测到相应的开关拨上，输入密码
    begin
        Code[Code_Bit] <= 0;                //赋值
        if(Code_Bit <= 3)
        begin
            Code_Bit <= Code_Bit + 1;        //指向下一位密码
            Count_CLK <= tens;
        end
    end
end
```



```

else
    begin
        Code[Code_Bit] <= 10;           //如果已输满四位，下一位置 10（肯定错误
的值）
        Code_Bit <= 4;                 //位数置 4，便于退格操作
    end
end

else if(SW1)
    begin
        Code[Code_Bit] <= 1;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW2)
    begin
        Code[Code_Bit] <= 2;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW3)
    begin
        Code[Code_Bit] <= 3;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;

```

```
        Count_CLK <= tens;
    end
else
    begin
        Code[Code_Bit] <= 10;
        Code_Bit <= 4;
    end
end

else if(SW4)
    begin
        Code[Code_Bit] <= 4;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW5)
    begin
        Code[Code_Bit] <= 5;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW6)
    begin
        Code[Code_Bit] <= 6;
        if(Code_Bit <= 3)
            begin
```

```

        Code_Bit <= Code_Bit + 1;
        Count_CLK <= tens;
    end
else
    begin
        Code[Code_Bit] <= 10;
        Code_Bit <= 4;
    end
end

else if(SW7)
    begin
        Code[Code_Bit] <= 7;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW8)
    begin
        Code[Code_Bit] <= 8;
        if(Code_Bit <= 3)
            begin
                Code_Bit <= Code_Bit + 1;
                Count_CLK <= tens;
            end
        else
            begin
                Code[Code_Bit] <= 10;
                Code_Bit <= 4;
            end
        end
    end

else if(SW9)
    begin
        Code[Code_Bit] <= 9;
        if(Code_Bit <= 3)

```

```
begin
    Code_Bit <= Code_Bit + 1;
    Count_CLK <= tens;
end
else
begin
    Code[Code_Bit] <= 10;
    Code_Bit <= 4;
end
end
end
```

C. 退格键：在输入状态或管理员状态下，按下上方按钮，即可抹去上一位输入的数字，等待重新输入该位

```
else if(BACKSPACE) //退格按键的相应操作
begin
    if(Code_Bit == 4)
begin
    Code_Bit <= 3; //退格指向上一位
    Code[3] <= 10; //将原本输入的数字 10，显示_
    Count_CLK <= tens;
end
    else if(Code_Bit == 3)
begin
    Code_Bit <= 2;
    Code[2] <= 10;
    Count_CLK <= tens;
end
    else if(Code_Bit == 2)
begin
    Code_Bit <= 1;
    Code[1] <= 10;
    Count_CLK <= tens;
end
    else if(Code_Bit == 1)
begin
    Code_Bit <= 0;
    Code[0] <= 10;
    Count_CLK <= tens;
end
    else
begin
    Code[0] <= 10; //如果已退完，则一直将第一位置 10 即可
    Count_CLK <= tens;
end
end
```

```
end
```

D. 确认键：右侧按钮

a) 在输入状态下，按下确认键，系统开始将输入的数字与密码进行比较，从而跳转到下一个状态；

```
else if(OK)                                //输入状态下按下确认按键，开始进行比较
begin
    Current_State <= COMPARE;
    Count_CLK <= tens;
end
else
    Count_CLK <= Count_CLK - 1;           //倒计时操作
end
```

b) 开锁状态下按下确认键，系统回到等待状态；

```
if(OK)                                     //按下确认回到等待状态
begin
    Current_State <= WAIT;
end
```

c) 在管理员状态下按下确认键，密码修改完毕。

```
else if(OK)                                //管理员状态下按下确认，将输入的数字置为
密码
begin
    Key[0] <= Code[0];
    Key[1] <= Code[1];
    Key[2] <= Code[2];
    Key[3] <= Code[3];
    Current_State <= WAIT;
end
```

E. 管理员按键：下方按键。

a) 在报警状态下按下该键，报警解除，蜂鸣器关闭，回到等待状态；

```
if(MANAGER)                                //按下管理员按键，报警解除
begin
    Error_Times <= 0;
    Current_State <= WAIT;
    buzzer_enable <= 0;                    // 立即关闭蜂鸣器
    Count_ALARM <= 0;                     // 重置计时器
end
end
```

b) 在其他状态下按下该键，开始修改密码；

4. 显示模块：

显示模块主要负责段码的显示。根据状态的不同，段码会有不同的显示。

A. 等待状态下，左侧四位显示 _ _ _ _；

```
if(Current_State == WAIT)           //等待状态下的显示
begin
case(num4)
0:begin
AN <= 8'b01111111;
Disp <= 4'ha;           //a 表示显示_
end
1:begin
AN <= 8'b10111111;
Disp <= 4'ha;
end
2:begin
AN <= 8'b11011111;
Disp <= 4'ha;
end
3:begin
AN <= 8'b11101111;
Disp <= 4'ha;
end
endcase
end
```

B. 输入密码状态下，根据拨动开关输入的数字，数码管左侧四位依次显示，未输入的位数仍显示；同时在输入状态下，最右侧一位数码管显示数字，代表输错的次数；

```
else if(Current_State == INPUT)      //输入状态下的显示
begin
case(num5)
0:begin
AN <= 8'b01111111;
Disp <= Code0;
end
1:begin
AN <= 8'b10111111;
```

```

        Disp <= Code1;
    end
2:begin
    AN <= 8'b11011111;
    Disp <= Code2;
end
3:begin
    AN <= 8'b11101111;
    Disp <= Code3;
end
4:begin
    AN <= 8'b11111110;
    Disp <= Error_Times;           //最右侧一位显示错误次数
end
endcase
end

```

C. 若密码正确，左起五位数码管显示“HELLO”；

```

else if(Current_State == UNLOCK)           //开锁状态下的显示
begin
    case(num5)
        0:begin
            AN <= 8'b01111111;
            Disp <= 4'hb;                   //b 表示显示 H
        end
        1:begin
            AN <= 8'b10111111;
            Disp <= 4'hc;                   //c 表示显示 E
        end
        2:begin
            AN <= 8'b11011111;
            Disp <= 4'hd;                   //d 表示显示 L
        end
        3:begin
            AN <= 8'b11101111;
            Disp <= 4'hd;
        end
        4:begin
            AN <= 8'b11110111;
            Disp <= 4'h0;
        end
    endcase
end

```

D. 若密码错误 1 次或者 2 次，左起五位数码显示“ERROR”；

```
else if(Current_State == ERROR)    //错误状态下的显示
begin
case(num5)
0:begin
AN <= 8'b01111111;
Disp <= 4'hc;                //c 表示显示 E
end
1:begin
AN <= 8'b10111111;
Disp <= 4'he;                //e 表示显示 R
end
2:begin
AN <= 8'b11011111;
Disp <= 4'he;
end
3:begin
AN <= 8'b11101111;
Disp <= 4'h0;
end
4:begin
AN <= 8'b11110111;
Disp <= 4'he;
end
endcase
end
```

E. 若密码错误 3 次，左侧四位数码管显示“EEEE”；

```
else if(Current_State == ALARM)    //报警状态下的显示
begin
case(num4)
0:begin
AN <= 8'b01111111;
Disp <= 4'hc;                //c 表示显示 e
end
1:begin
AN <= 8'b10111111;
Disp <= 4'hc;
end
2:begin
AN <= 8'b11011111;
Disp <= 4'hc;
end
3:begin
```



```

        AN <= 8'b11101111;
        Disp <= 4'hc;
    end
endcase
end

```

F. 管理员状态下，数码管显示_ _ _ _，并等待拨码开关的输入从而依次进行显示。

```

else if(Current_State == ADMIN)           //管理员状态下的显示
    begin
        case(num4)
            0:begin
                AN <= 8'b01111111;
                Disp <= Code0;
            end
            1:begin
                AN <= 8'b10111111;
                Disp <= Code1;
            end
            2:begin
                AN <= 8'b11011111;
                Disp <= Code2;
            end
            3:begin
                AN <= 8'b11101111;
                Disp <= Code3;
            end
        endcase
    end
end

```

5. 三色 LED 显示模块:

为了更直观的指示当前状态，我们设置了三色 LED 在不同状态下的不同显示。

A. 等待状态下，两个三色 LED 灯（下称 RGB）均显示蓝色，且亮 5s 灭 3s 依次循环进行；

```

if(Current_State == WAIT)           //等待状态两个均为蓝色
    begin
        Count_RGBW <= 0;
        if(Count_RGBW < 500000000)   //利用计数器实现呼吸显示
            begin
                Count_RGBW <= Count_RGBW + 1;
                RGB1_RED <= 1'b0;
                RGB1_GREEN <= 1'b0;
                RGB1_BLUE <= 1'b1;
            end
        end
    end

```

```
        RGB2_RED <= 1'b0;
        RGB2_GREEN <= 1'b0;
        RGB2_BLUE <= 1'b1;
    end
else if(Count_RGBW == 800000000)
    Count_RGBW <= 0;
else
    begin
        Count_RGBW <= Count_RGBW + 1;
        RGB1_RED <= 1'b0;
        RGB1_GREEN <= 1'b0;
        RGB1_BLUE <= 1'b0;
        RGB2_RED <= 1'b0;
        RGB2_GREEN <= 1'b0;
        RGB2_BLUE <= 1'b0;
    end
end
end
```

B. 输入状态下，左侧 RGB 显示绿色，右侧 RGB 显示红色（表示将产生正确或者错误的两种结果）；

```
else if(Current_State == INPUT)           //输入状态一个为红色，一个为绿色
    begin
        RGB1_RED <= 1'b1;
        RGB1_GREEN <= 1'b0;
        RGB1_BLUE <= 1'b0;
        RGB2_RED <= 1'b0;
        RGB2_GREEN <= 1'b1;
        RGB2_BLUE <= 1'b0;
    end
end
```

C. 开锁状态下，两个 RGB 显示绿色（正确）；

```
else if(Current_State == UNLOCK)          //开锁状态两个均为绿色
    begin
        RGB1_RED <= 1'b0;
        RGB1_GREEN <= 1'b1;
        RGB1_BLUE <= 1'b0;
        RGB2_RED <= 1'b0;
        RGB2_GREEN <= 1'b1;
        RGB2_BLUE <= 1'b0;
    end
end
```

D. 错误状态下，两个 RGB 显示黄色（用于警示作用）；

```
else if(Current_State == ERROR)           //错误状态两个均为黄色
    begin
        RGB1_RED <= 1'b1;
```

```

        RGB1_GREEN <= 1'b1;
        RGB1_BLUE <= 1'b0;
        RGB2_RED <= 1'b1;
        RGB2_GREEN <= 1'b1;
        RGB2_BLUE <= 1'b0;
    end

```

E. 报警状态下，两个 RGB 显示红色，并和上方设计的 10 个 LED 同步以 1Hz 的频率进行闪烁报警；

```

else if(Current_State == ALARM)           //报警状态两个均为红色
    begin
        Count_RGBA <= 0;                  //利用计数器实现闪烁显示
        if(Count_RGBA < 50000000)
            begin
                Count_RGBA <= Count_RGBA + 1;
                RGB1_RED <= 1'b1;
                RGB1_GREEN <= 1'b0;
                RGB1_BLUE <= 1'b0;
                RGB2_RED <= 1'b1;
                RGB2_GREEN <= 1'b0;
                RGB2_BLUE <= 1'b0;
            end
        else if(Count_RGBA == 100000000)
            Count_RGBA <= 0;
        else
            begin
                Count_RGBA <= Count_RGBA + 1;
                RGB1_RED <= 1'b0;
                RGB1_GREEN <= 1'b0;
                RGB1_BLUE <= 1'b0;
                RGB2_RED <= 1'b0;
                RGB2_GREEN <= 1'b0;
                RGB2_BLUE <= 1'b0;
            end
        end
    end
end

```

F. 管理员状态下，两个 RGB 显示白色（表示该状态下密码已成白纸待写）。

```

else if(Current_State == ADMIN)           //管理员状态两个均为白色
    begin
        RGB1_RED <= 1'b1;
        RGB1_GREEN <= 1'b1;
        RGB1_BLUE <= 1'b1;
        RGB2_RED <= 1'b1;
        RGB2_GREEN <= 1'b1;
        RGB2_BLUE <= 1'b1;
    end

```

```
end
```

6. 蓝牙模块:

为了使得物理按键信号与蓝牙信号相匹配, 并且较强的兼容信号, 只需要在 TOP 层中添加信号融合逻辑, 使得蓝牙和物理按键可以同时工作, 并且任一方式触发都能正常工作。

(1) UART 通信接收器

波特率配置: 支持 9600 bps 标准波特率 (基于 100MHz 系统时钟)

异步串行通信: 实现标准的 8-N-1 格式 (8 位数据位, 无奇偶校验, 1 位停止位)

三级同步器: 采用三级触发器对异步输入信号进行同步处理, 提高系统稳定性

状态机控制: 使用有限状态机实现 UART 接收时序控制, 包括空闲、起始位检测、数据接收和停止位验证四个状态

```
module BLUETOOTH_INTERFACE(  
    input CLK,  
    input RESET,  
    input RX,  
    output TX,  
  
    // 输出单脉冲信号  
    output reg BACKSPACE_BT,  
    output reg OK_BT,  
    output reg MANAGER_BT,  
    output reg START_BT,  
    output reg SW0_BT, SW1_BT, SW2_BT, SW3_BT, SW4_BT,  
    output reg SW5_BT, SW6_BT, SW7_BT, SW8_BT, SW9_BT,  
  
    // 状态输入 (用于反馈)  
    input [2:0] CURRENT_STATE,  
    input [2:0] ERROR_TIMES,  
    input [9:0] LED_STATUS,  
  
    // 调试输出  
    output reg [7:0] DEBUG_LAST_DATA,  
    output DEBUG_RX_DONE  
);
```

定义模块名称和所有输入输出端口

CLK, RESET, RX 为基本控制信号, 其中 RX 接收蓝牙数据

14 个输出信号 (BACKSPACE_BT 到 SW9_BT) 对应不同的控制命令

状态输入用于系统反馈 (虽然在当前代码中未使用)

调试输出便于开发过程中监控模块工作状态

(2) 内部寄存器和信号声明

```
// 接收状态机
```

```

reg [3:0] rx_state;
reg [15:0] rx_counter;
reg [3:0] rx_bit_count;
reg [7:0] rx_data;
reg [7:0] rx_data_reg;
reg rx_done;

// 同步寄存器
reg rx_sync1, rx_sync2, rx_sync3;

// UART 接收状态定义
parameter RX_IDLE = 4'b0000;
parameter RX_START = 4'b0001;
parameter RX_DATA = 4'b0010;
parameter RX_STOP = 4'b0011;

assign TX = 1'b1;
assign DEBUG_RX_DONE = rx_done;

```

rx_state: 状态机当前状态寄存器
 rx_counter: 波特率计数器, 用于时序控制
 rx_bit_count: 数据位计数器 (0-7)
 rx_data: 正在接收的数据缓存
 rx_data_reg: 接收完成的数据寄存器
 rx_done: 接收完成标志, 产生单周期脉冲
 rx_sync1/2/3: 三级同步寄存器链, 用于跨时钟域同步
 四个状态常量定义了 UART 接收的四个阶段
 (3) 三级同步器实现

```

// 三级同步
always @(posedge CLK) begin
    if (RESET) begin
        rx_sync1 <= 1'b1;
        rx_sync2 <= 1'b1;
        rx_sync3 <= 1'b1;
    end else begin
        rx_sync1 <= RX;
        rx_sync2 <= rx_sync1;
        rx_sync3 <= rx_sync2;
    end
end

```

解决异步信号 RX 与系统时钟 CLK 之间的同步问题

三级寄存器形成移位寄存器链: $RX \rightarrow rx_sync1 \rightarrow rx_sync2 \rightarrow rx_sync3$

复位时全部置 1 (UART 空闲状态为高电平)

有效减少亚稳态问题, 提高系统可靠性

(4) UART 接收状态机 - 主控制逻辑

①状态机总体结构

```
// UART 接收逻辑
always @(posedge CLK) begin
    if (RESET) begin
        rx_state <= RX_IDLE;
        rx_counter <= 0;
        rx_bit_count <= 0;
        rx_data <= 0;
        rx_data_reg <= 0;
        rx_done <= 0;
        DEBUG_LAST_DATA <= 0;
    end else begin
        rx_done <= 0;
    end
end
```

②RX_IDLE 状态（空闲状态）RX_START 状态（起始位验证）RX_DATA 状态（数据位接收）RX_STOP 状态（停止位验证与数据输出）

```
case (rx_state)
    RX_IDLE: begin
        rx_counter <= 0;
        if (!rx_sync3) begin
            rx_state <= RX_START;
            rx_counter <= 0;
        end
    end

    RX_START: begin
        rx_counter <= rx_counter + 1;
        if (rx_counter >= (BAUD_DIV/2)) begin
            if (!rx_sync3) begin
                rx_state <= RX_DATA;
                rx_counter <= 0;
                rx_bit_count <= 0;
                rx_data <= 8'h00;
            end else begin
                rx_state <= RX_IDLE;
            end
        end
    end

    RX_DATA: begin
        rx_counter <= rx_counter + 1;
        if (rx_counter >= BAUD_DIV) begin
            rx_counter <= 0;
            if (rx_bit_count >= 7) begin
```

```

        rx_state <= RX_STOP;
    end else begin
        rx_bit_count <= rx_bit_count + 1;
    end
end else if (rx_counter == (BAUD_DIV/2)) begin
    rx_data[rx_bit_count] <= rx_sync3;
end
end
end

RX_STOP: begin
    rx_counter <= rx_counter + 1;
    if (rx_counter == (BAUD_DIV/2)) begin
        if (rx_sync3) begin
            rx_data_reg <= rx_data;
            DEBUG_LAST_DATA <= rx_data;
            rx_done <= 1; // 单周期脉冲
        end
    end
end

    if (rx_counter >= BAUD_DIV) begin
        rx_state <= RX_IDLE;
        rx_counter <= 0;
    end
end

    default: rx_state <= RX_IDLE;
endcase
end
end

```

RX_IDLE 状态（空闲状态）：等待起始位（低电平）的到来，检测到起始位后转入 RX_START 状态，清零计数器准备时序测量

RX_START 状态（起始位验证）：等待半个波特率周期后再次检测起始位，在起始位中央采样，确认是否为有效起始位，如果确认有效，转入数据接收状态并初始化相关寄存器，如果起始位无效（可能是噪声），返回空闲状态。

RX_DATA 状态（数据位接收）：接收 8 个数据位（LSB 先传输），在每个数据位的中央时刻采样（BAUD_DIV/2），使用 rx_bit_count 跟踪当前接收的位数，接收完 8 位后转入停止位检测状态

RX_STOP 状态（停止位验证与数据输出）：验证停止位（应为高电平），如果停止位有效，将接收到的数据存入 rx_data_reg，同时更新调试寄存器 DEBUG_LAST_DATA，产生单周期的 rx_done 脉冲信号，完成一个字节的接收后返回空闲状态。

（5）命令解析与单脉冲输出

// 单脉冲生成 - 关键改进

```
always @(posedge CLK) begin
    if (RESET) begin
        {BACKSPACE_BT, OK_BT, MANAGER_BT, START_BT} <= 4'b0;
        {SW0_BT, SW1_BT, SW2_BT, SW3_BT, SW4_BT} <= 5'b0;
        {SW5_BT, SW6_BT, SW7_BT, SW8_BT, SW9_BT} <= 5'b0;
    end else begin
        // 默认清零所有输出（确保只有单周期脉冲）
        {BACKSPACE_BT, OK_BT, MANAGER_BT, START_BT} <= 4'b0;
        {SW0_BT, SW1_BT, SW2_BT, SW3_BT, SW4_BT} <= 5'b0;
        {SW5_BT, SW6_BT, SW7_BT, SW8_BT, SW9_BT} <= 5'b0;

        // 只在接收完成的那一个时钟周期输出脉冲
        if (rx_done) begin
            case (rx_data_reg)
                8'h53, 8'h73: START_BT <= 1;      // 'S' or 's' - 单周期脉冲
                8'h4F, 8'h6F: OK_BT <= 1;        // 'O' or 'o'
                8'h4D, 8'h6D: MANAGER_BT <= 1;    // 'M' or 'm'
                8'h42, 8'h62: BACKSPACE_BT <= 1;  // 'B' or 'b'
                8'h30: SW0_BT <= 1;               // '0'
                8'h31: SW1_BT <= 1;               // '1'
                8'h32: SW2_BT <= 1;               // '2'
                8'h33: SW3_BT <= 1;               // '3'
                8'h34: SW4_BT <= 1;               // '4'
                8'h35: SW5_BT <= 1;               // '5'
                8'h36: SW6_BT <= 1;               // '6'
                8'h37: SW7_BT <= 1;               // '7'
                8'h38: SW8_BT <= 1;               // '8'
                8'h39: SW9_BT <= 1;               // '9'
                default: begin
                    // 无效命令，保持清零状态
                end
            endcase
        end
    end
end
```

①是整个模块的输出控制逻辑，确保产生标准的单脉冲信号，关键设计：每个时钟周期都先清零所有输出，再根据条件置位只有在 rx_done 有效时才进行命令解析和输出

②命令映射：字母命令：S/s(启动)、O/o(确认)、M/m(管理)、B/b(退格)，数字命令：0-9 对应相应的开关信号，支持大小写字母（提高用户友好性）

单脉冲保证：由于每周期都清零，输出信号只持续一个时钟周期

无效命令不产生任何输出，提高系统安全性

定义的输入输出信号：


```

module TOP(
    input CLK,

    // 物理输入信号
    input SW0_IN,input SW1_IN,input SW2_IN,input SW3_IN,input SW4_IN,
    input SW5_IN,input SW6_IN,input SW7_IN,input SW8_IN,input SW9_IN,
    input RESET,
    input BACKSPACE_IN,
    input OK_IN,
    input MANAGER_IN,
    input START_IN,

    // 蓝牙接口信号
    input BT_RX,
    output BT_TX,

    // 输出信号
    output [2:0] STATE,
    output [7:0] AN,
    output [7:0] SEG,
    output [9:0] LED,
    output RGB1_RED, output RGB1_BLUE, output RGB1_GREEN,
    output RGB2_RED, output RGB2_BLUE, output RGB2_GREEN,

    output BUZZER // 新增蜂鸣器输出
);

// 物理按键消抖后的信号
wire BACKSPACE_PHYSICAL;
wire OK_PHYSICAL;
wire MANAGER_PHYSICAL;
wire START_PHYSICAL;
wire SW0_PHYSICAL, SW1_PHYSICAL, SW2_PHYSICAL, SW3_PHYSICAL, SW4_PHYSICAL;
wire SW5_PHYSICAL, SW6_PHYSICAL, SW7_PHYSICAL, SW8_PHYSICAL, SW9_PHYSICAL;

// 蓝牙输出信号
wire BACKSPACE_BT;
wire OK_BT;
wire MANAGER_BT;
wire START_BT;
wire SW0_BT, SW1_BT, SW2_BT, SW3_BT, SW4_BT;
wire SW5_BT, SW6_BT, SW7_BT, SW8_BT, SW9_BT;

// 最终组合信号（物理输入 OR 蓝牙输入）

```

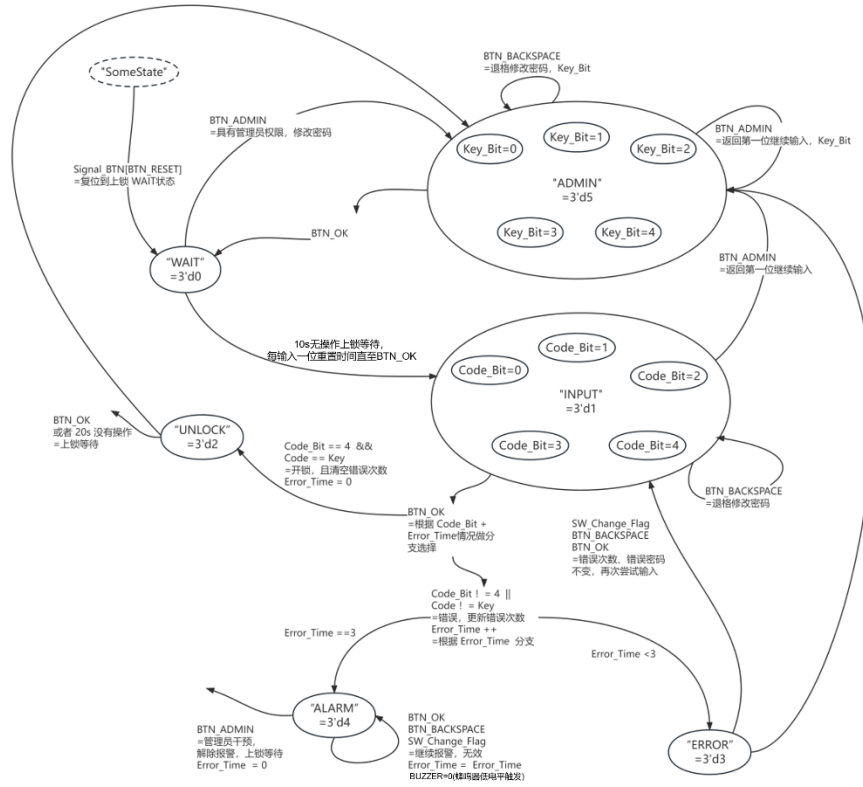
```
wire BACKSPACE = BACKSPACE_PHYSICAL | BACKSPACE_BT;
wire OK = OK_PHYSICAL | OK_BT;
wire MANAGER = MANAGER_PHYSICAL | MANAGER_BT;
wire START = START_PHYSICAL | START_BT;

wire SW0 = SW0_PHYSICAL | SW0_BT;
wire SW1 = SW1_PHYSICAL | SW1_BT;
wire SW2 = SW2_PHYSICAL | SW2_BT;
wire SW3 = SW3_PHYSICAL | SW3_BT;
wire SW4 = SW4_PHYSICAL | SW4_BT;
wire SW5 = SW5_PHYSICAL | SW5_BT;
wire SW6 = SW6_PHYSICAL | SW6_BT;
wire SW7 = SW7_PHYSICAL | SW7_BT;
wire SW8 = SW8_PHYSICAL | SW8_BT;
wire SW9 = SW9_PHYSICAL | SW9_BT;

// 蓝牙活动指示 - 任何蓝牙信号激活时为高
assign BT_ACTIVE = BACKSPACE_BT | OK_BT | MANAGER_BT | START_BT |
                   SW0_BT | SW1_BT | SW2_BT | SW3_BT | SW4_BT |
                   SW5_BT | SW6_BT | SW7_BT | SW8_BT | SW9_BT;
```

模块/进程间的关系

- A. 等待状态下，左侧四位显示，两个三色 LED 灯（下称 RGB）均显示蓝色，且亮 5s 灭 3s 依次循环进行；
- B. 输入密码状态下，根据拨动开关输入的数字，数码管左侧四位依次显示，未输入的位数仍显示，同时在输入状态下，最右侧一位数码管显示数字，代表输错的次数，左侧 RGB 显示绿色，右侧 RGB 显示红色（表示将产生正确或者错误的两种结果）；
- C. 若密码正确，左起五位数码管显示“HELLO”，两个 RGB 显示绿色（正确）；
- D. 若密码错误 1 次或者 2 次，左起五位数码管显示“ERROR”，两个 RGB 显示黄色（用于警示作用）；
- E. 若密码错误 3 次，左侧四位数码管显示“EEEE”，两个 RGB 显示白色（表示该状态下密码已成白纸待写）；
- F. 管理员状态下，数码管显示 _ _ _ _，并等待拨码开关的输入从而依次进行显示。



状态图

硬件接口对应：

1. 物理硬件接口对应

按键	名称	端口	功能
SW0	SW0_IN	J15	输入数字0
SW1	SW1_IN	L16	输入数字1
SW2	SW2_IN	M13	输入数字2
SW3	SW3_IN	R15	输入数字3
SW4	SW4_IN	R17	输入数字4
SW5	SW5_IN	T18	输入数字5
SW6	SW6_IN	U18	输入数字6
SW7	SW7_IN	R13	输入数字7
SW8	SW8_IN	T8	输入数字8
SW9	SW9_IN	U8	输入数字9
BTNC	RESET	N17	重置键
BTNU	BACKSPACE_IN	M18	退格键
BTNL	START_IN	P17	开始键
BTNR	OK_IN	M17	确认键
BTND	MANAGER_IN	P18	管理员按键

三. 实验结果及结论

（包括仿真方法和仿真文件、仿真结果、演示结果简单描述、存在问题或后续可改进之处，

以及必要的实验结论。)

1. 仿真方法和仿真文件

由于系统全部功能实现仿真时间过长，我们只做主控模块的仿真，验证状态转换的正确性操作如下：

仿真过程

```
module CTRL_INTEGRATION_TB();
    // 时钟和复位
    reg CLK;
    reg RESET;

    // 模拟物理按键消抖后的信号
    reg BACKSPACE_PHYSICAL;
    reg OK_PHYSICAL;
    reg MANAGER_PHYSICAL;
    reg START_PHYSICAL;
    reg SW0_PHYSICAL, SW1_PHYSICAL, SW2_PHYSICAL, SW3_PHYSICAL, SW4_PHYSICAL;
    reg SW5_PHYSICAL, SW6_PHYSICAL, SW7_PHYSICAL, SW8_PHYSICAL, SW9_PHYSICAL;

    // 模拟蓝牙输出信号
    reg BACKSPACE_BT;
    reg OK_BT;
    reg MANAGER_BT;
    reg START_BT;
    reg SW0_BT, SW1_BT, SW2_BT, SW3_BT, SW4_BT;
    reg SW5_BT, SW6_BT, SW7_BT, SW8_BT, SW9_BT;
```

这部分代码定义了测试平台的基础架构。测试台模拟了一个双输入系统，其中包含物理按键输入和蓝牙无线输入两套完全独立的信号源。所有的物理按键信号使用_PHYSICAL 后缀标识，而蓝牙信号使用_BT 后缀标识。这种设计允许测试台验证系统在接收来自不同输入源的命令时是否能够正确响应，同时也为后续的信号组合逻辑提供了基础。

```
// 组合信号（模拟 TOP 模块中的 OR 逻辑）
    wire BACKSPACE = BACKSPACE_PHYSICAL | BACKSPACE_BT;
    wire OK = OK_PHYSICAL | OK_BT;
    wire MANAGER = MANAGER_PHYSICAL | MANAGER_BT;
    wire START = START_PHYSICAL | START_BT;

    wire SW0 = SW0_PHYSICAL | SW0_BT;
    wire SW1 = SW1_PHYSICAL | SW1_BT;
    wire SW2 = SW2_PHYSICAL | SW2_BT;
    wire SW3 = SW3_PHYSICAL | SW3_BT;
    wire SW4 = SW4_PHYSICAL | SW4_BT;
    wire SW5 = SW5_PHYSICAL | SW5_BT;
    wire SW6 = SW6_PHYSICAL | SW6_BT;
```

```

wire SW7 = SW7_PHYSICAL | SW7_BT;
wire SW8 = SW8_PHYSICAL | SW8_BT;
wire SW9 = SW9_PHYSICAL | SW9_BT;

```

这里实现了关键的信号融合逻辑。每个功能信号都通过 OR 门将物理输入和蓝牙输入进行逻辑或运算，这意味着无论是物理按键触发还是蓝牙命令触发，系统都会产生相同的响应。这种设计确保了用户可以通过任何一种方式控制系统，实现了真正的多模态输入。OR 逻辑的选择非常合理，因为它允许任何一个输入源激活相应的功能，同时不会产生冲突。

```

task press_physical;
    input [3:0] key_type; // 0-9=SW, 10=START, 11=OK, 12=MANAGER, 13=BACKSPACE
    begin
        case (key_type)
            0: begin SW0_PHYSICAL = 1; #10; SW0_PHYSICAL = 0; $display("物理按键
SW0"); end
            1: begin SW1_PHYSICAL = 1; #10; SW1_PHYSICAL = 0; $display("物理按键
SW1"); end
            // ... 其他按键
            10: begin START_PHYSICAL = 1; #10; START_PHYSICAL = 0; $display("物理
按键 START"); end
            11: begin OK_PHYSICAL = 1; #10; OK_PHYSICAL = 0; $display("物理按键
OK"); end
            12: begin MANAGER_PHYSICAL = 1; #10; MANAGER_PHYSICAL = 0; $display("
物理按键 MANAGER"); end
            13: begin BACKSPACE_PHYSICAL = 1; #10; BACKSPACE_PHYSICAL = 0;
$display("物理按键 BACKSPACE"); end
        endcase
        #100; // 等待处理
    end
endtask

```

这个任务模拟了物理按键的按下和释放动作。每个按键操作持续 10ns 的时间，然后自动释放，这模拟了真实按键的瞬时脉冲特性。任务结束后额外等待 100ns，给系统足够的时间来处理按键事件。通过使用任务封装，测试代码变得更加简洁和可读，同时确保了所有物理按键操作的一致性。编码方案使用数字 0-9 表示相应的开关，10-13 表示特殊功能键，这种编码方式直观且易于理解。

```

task send_bluetooth;
    input [3:0] key_type; // 0-9=SW, 10=START, 11=OK, 12=MANAGER, 13=BACKSPACE
    begin
        case (key_type)
            0: begin SW0_BT = 1; #10; SW0_BT = 0; $display("蓝牙命令 SW0"); end
            1: begin SW1_BT = 1; #10; SW1_BT = 0; $display("蓝牙命令 SW1"); end
            // ... 其他命令
            10: begin START_BT = 1; #10; START_BT = 0; $display("蓝牙命令
START"); end
            11: begin OK_BT = 1; #10; OK_BT = 0; $display("蓝牙命令 OK"); end

```

```

12: begin MANAGER_BT = 1; #10; MANAGER_BT = 0; $display("蓝牙命令
MANAGER"); end
13: begin BACKSPACE_BT = 1; #10; BACKSPACE_BT = 0; $display("蓝牙命令
BACKSPACE"); end
endcase
#100; // 等待处理
end
endtask

```

蓝牙命令模拟任务与物理按键任务具有相同的结构和时序特性，但操作的是蓝牙信号线。这种对称设计确保了两种输入方式在时序上的一致性，便于进行比较测试。任务同样产生 10ns 的脉冲信号，模拟蓝牙模块接收到命令后产生的单脉冲输出。这种一致的时序设计使得测试结果更具可比性。

TEST1:

```

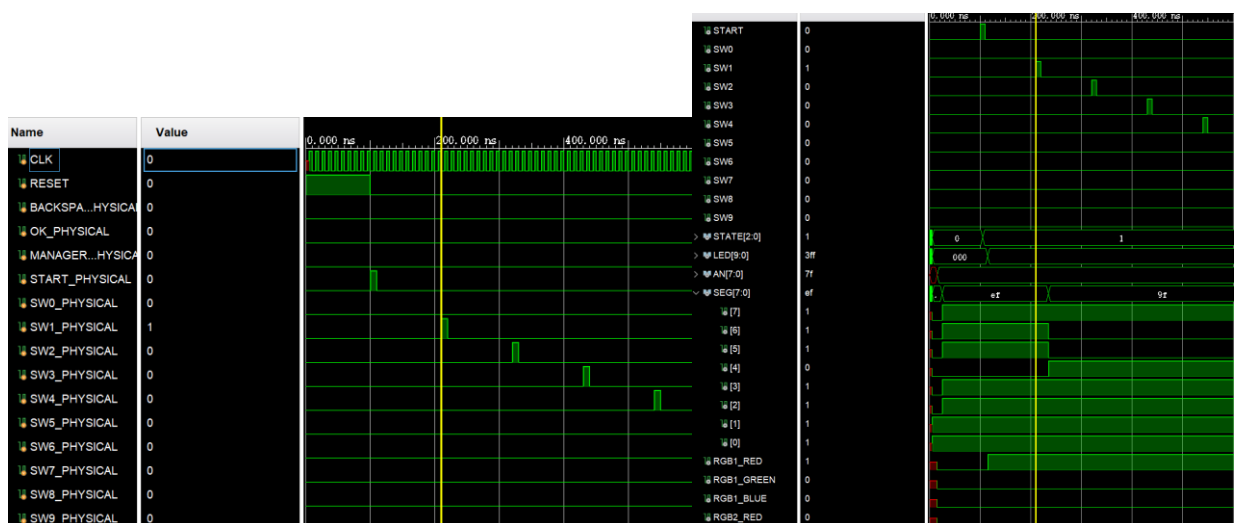
// 测试 1: 物理按键启动和密码输入
$display(" 测试 1: 物理按键完整流程");
press_physical(10); // START
show_status("物理 START 后");

press_physical(1); // SW1 - 第一位密码
press_physical(2); // SW2 - 第二位密码
press_physical(3); // SW3 - 第三位密码
press_physical(4); // SW4 - 第四位密码
press_physical(11); // OK
show_status("物理密码输入后");

```

第一个测试验证了纯物理按键操作的完整流程。测试模拟了用户使用物理按键启动系统、输入四位密码（1234）、然后确认的过程。这个测试确保了物理输入路径的完整性和控制逻辑的正确性。在每个关键步骤后都显示系统状态，便于跟踪状态变化和定位潜在问题。

其仿真结果如下：



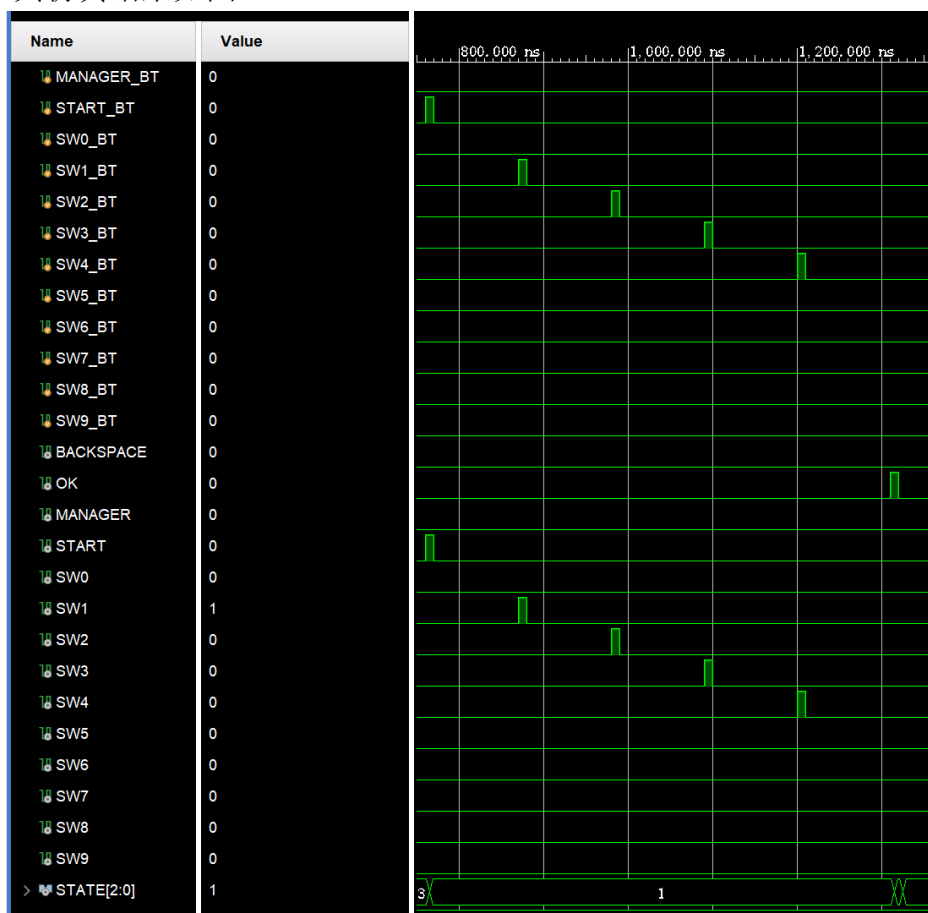
TEST2:

```
// 测试 2: 蓝牙命令启动和密码输入
$display(" 测试 2: 蓝牙命令完整流程");
send_bluetooth(10); // START
show_status("蓝牙 START 后");

send_bluetooth(1); // SW1 - 第一位密码
send_bluetooth(2); // SW2 - 第二位密码
send_bluetooth(3); // SW3 - 第三位密码
send_bluetooth(4); // SW4 - 第四位密码
send_bluetooth(11); // OK
show_status("蓝牙密码输入后");
```

第二个测试重复了与第一个测试相同的操作序列，但完全通过蓝牙命令执行。这个测试验证了蓝牙输入路径的功能完整性，并且可以与物理按键测试的结果进行对比，确认两种输入方式产生相同的系统响应。这种对称测试方法是验证多模态输入系统的标准做法。

其仿真结果如图：



TEST3:

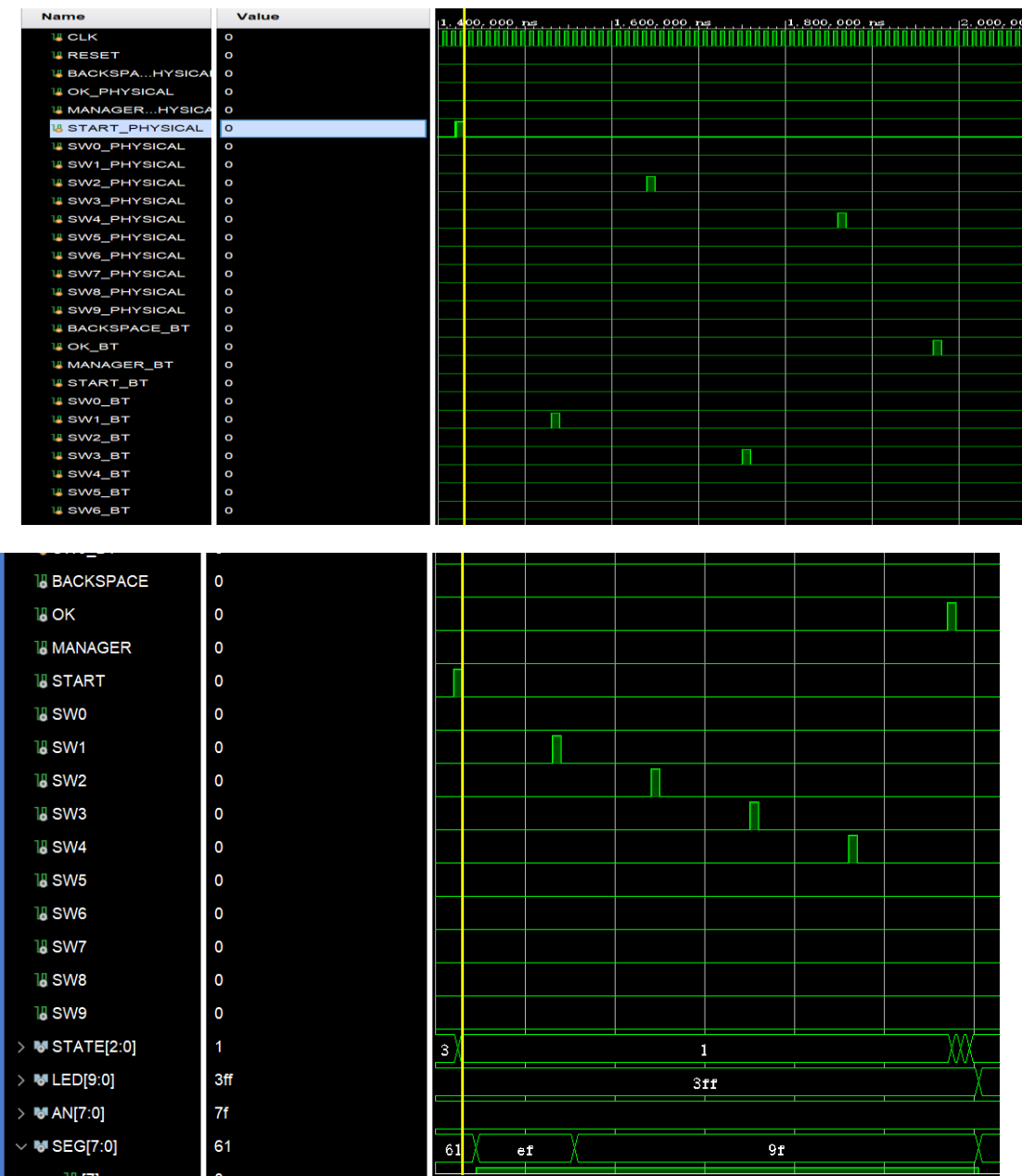
```
// 测试 3: 混合输入测试（关键测试）
$display("? 测试 3: 混合输入流程（物理+蓝牙）");
press_physical(10); // 物理 START
show_status("混合-物理 START");
```



```
send_bluetooth(1); // 蓝牙 SW1
press_physical(2); // 物理 SW2
send_bluetooth(3); // 蓝牙 SW3
press_physical(4); // 物理 SW4
send_bluetooth(11); // 蓝牙 OK
show_status("混合输入完成");
```

这是最重要的测试之一，验证了系统处理混合输入的能力。测试交替使用物理按键和蓝牙命令来完成一个完整的操作序列。这种测试模拟了真实使用场景中用户可能同时使用两种输入方式的情况。混合输入测试的成功表明信号组合逻辑工作正确，系统能够无缝地处理来自不同源的输入信号。

仿真结果如图：



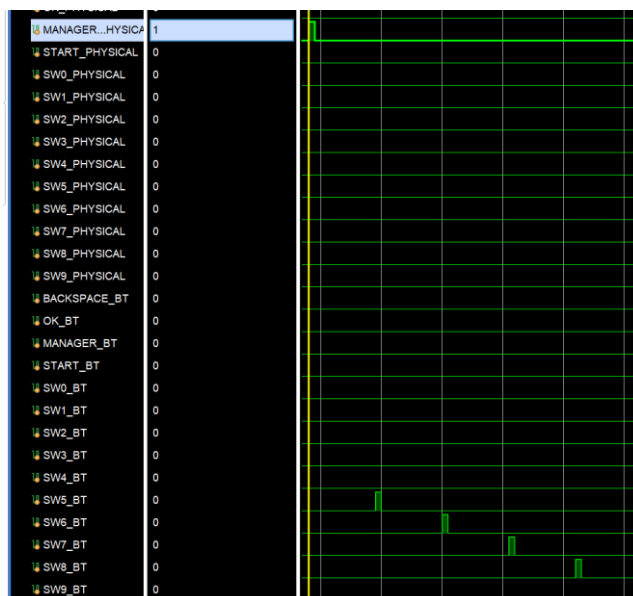
TEST4:

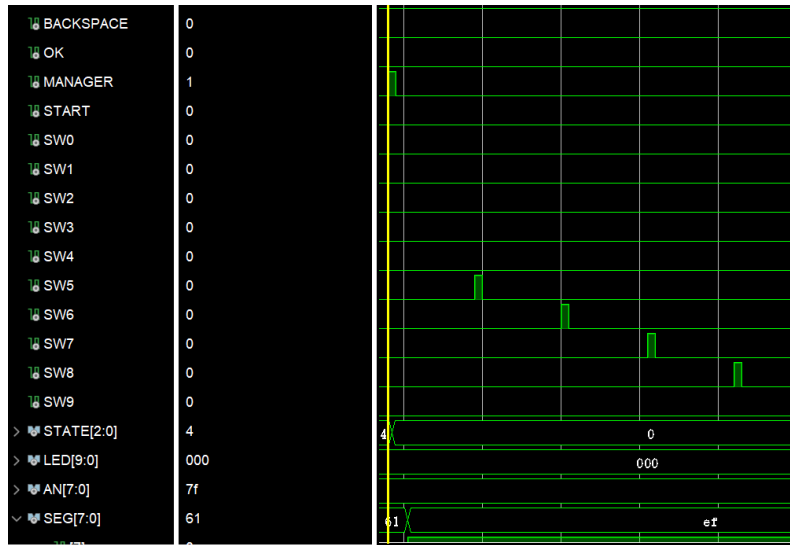
```
// 测试 4: 管理员功能测试
$display("测试 4: 管理员功能");
press_physical(12); // 物理 MANAGER
show_status("进入管理员模式");

send_bluetooth(5); // 新密码第一位
send_bluetooth(6); // 新密码第二位
send_bluetooth(7); // 新密码第三位
send_bluetooth(8); // 新密码第四位
press_physical(11); // 物理 OK 确认
show_status("密码修改完成");
```

管理员功能测试验证了系统的高级管理功能。测试首先通过物理按键进入管理员模式，然后使用蓝牙命令输入新密码（5678），最后用物理按键确认。这个测试不仅验证了管理员模式的基本功能，还再次证明了混合输入模式的有效性。新密码的选择（5678）与之前的密码（1234）不同，为后续的验证测试提供了基础。

仿真结果如下：



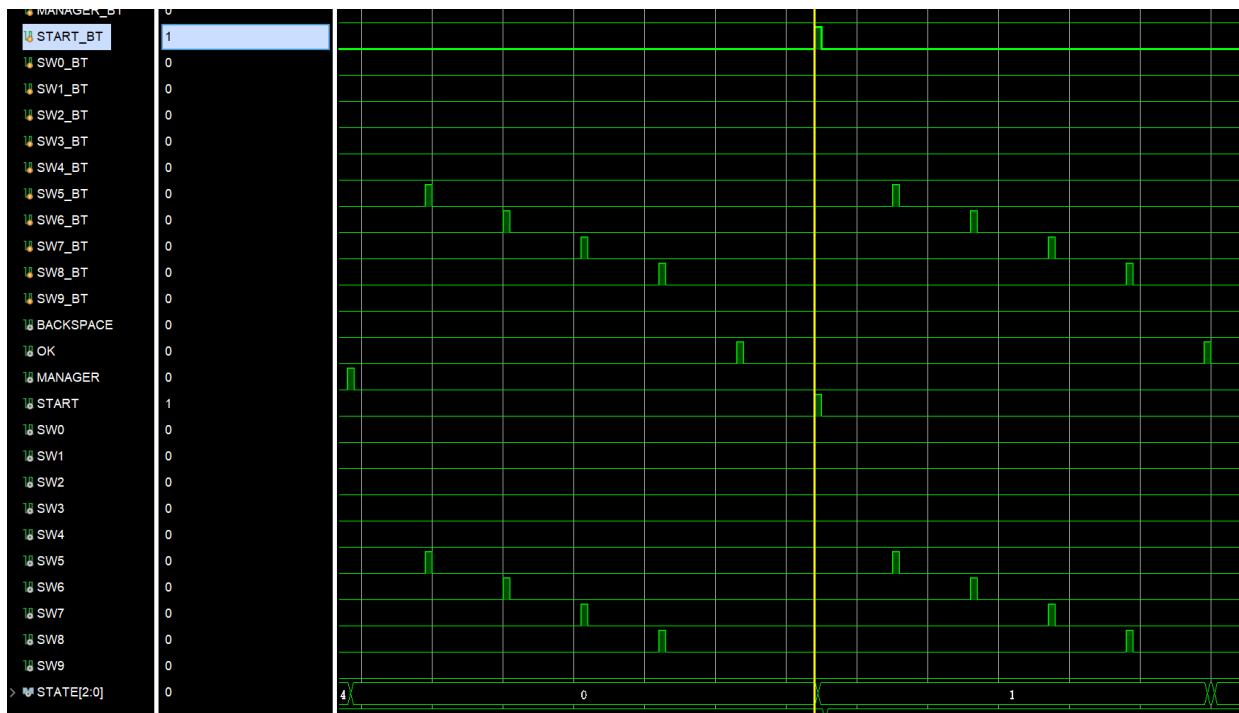


TEST5:

```
// 测试 5: 验证新密码
$display(" 测试 5: 验证新密码");
send_bluetooth(10); // START
send_bluetooth(5);  // 新密码
send_bluetooth(6);
send_bluetooth(7);
send_bluetooth(8);
send_bluetooth(11); // OK
show_status("新密码验证");
```

这个测试验证密码修改功能是否真正生效。使用新设置的密码（5678）通过蓝牙命令进行完整的认证流程。测试的成功表明系统不仅能够接受新密码的设置，还能够正确存储和验证新密码，证明了系统状态管理的正确性。

仿真结果如下：



TEST6:

// 测试 6: 信号组合验证

\$display(" 测试 6: 信号组合逻辑验证");

\$display("测试同时激活物理和蓝牙信号...");

// 同时激活物理和蓝牙 SW0

SW0_PHYSICAL = 1;

SW0_BT = 1;

#10;

if (SW0 === 1'b1)

\$display(" SW0 组合信号正确: PHYSICAL|BT = 1|1 = 1");

else

\$display(" SW0 组合信号错误");

SW0_PHYSICAL = 0;

#10;

if (SW0 === 1'b1)

\$display(" SW0 组合信号正确: PHYSICAL|BT = 0|1 = 1");

else

\$display(" SW0 组合信号错误");

SW0_BT = 0;

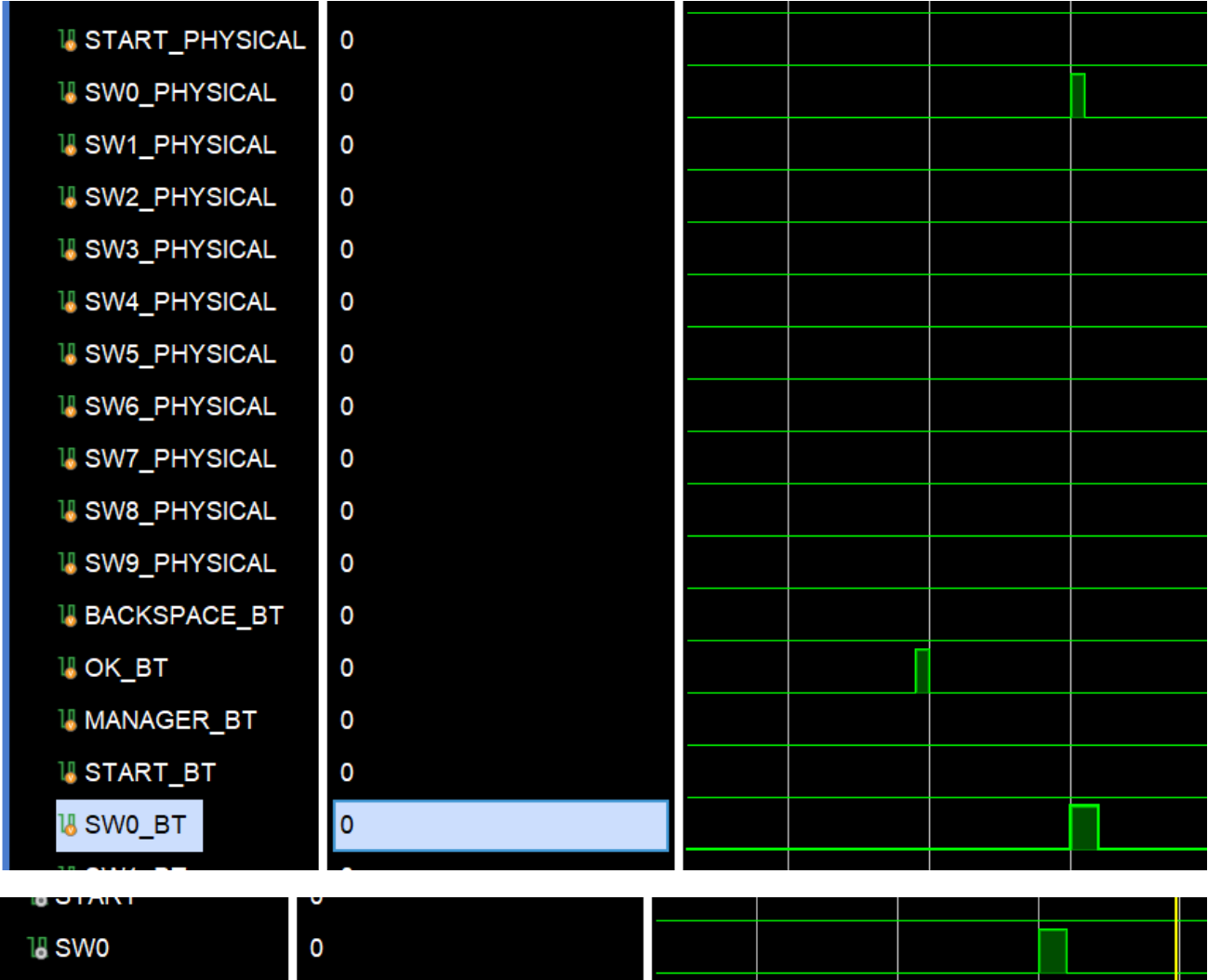
#10;

if (SW0 === 1'b0)

\$display("SW0 组合信号正确: PHYSICAL|BT = 0|0 = 0");

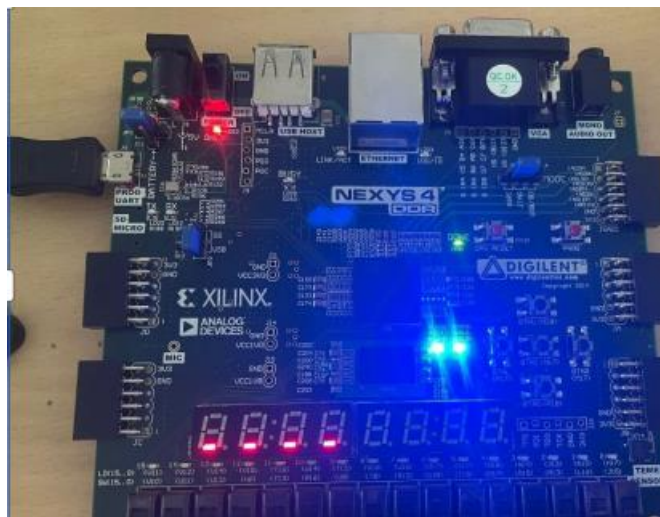
```
else
    $display(" SW0 组合信号错误");
```

这个测试专门验证 OR 门组合逻辑的正确性。通过逐步改变物理和蓝牙输入信号的状态，测试验证了所有可能的输入组合（1|1=1，0|1=1，0|0=0）都产生正确的输出。这种详细的逻辑验证确保了信号融合机制的可靠性，这对整个系统的正确运行至关重要。

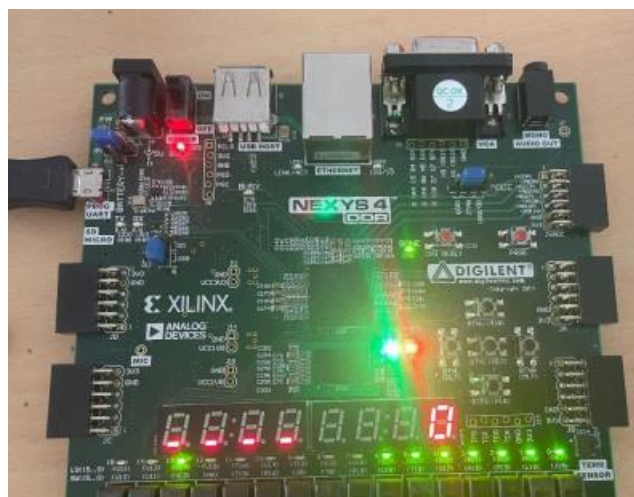


2. 实物操作如下：

初始状态（等待状态）：数码管显示_ _ _，三色 LED 为蓝色；



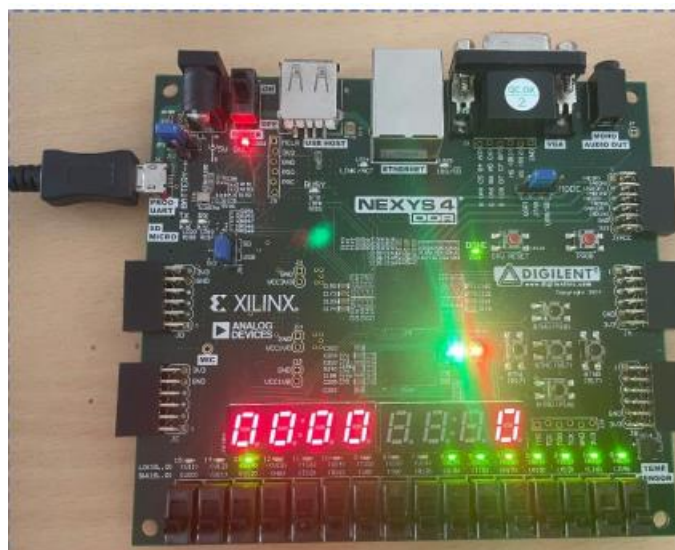
按下开始键，进入输入状态，三色 LED 灯一红一绿，下方 LED 开始倒计时，最右侧一位显示错误次数；



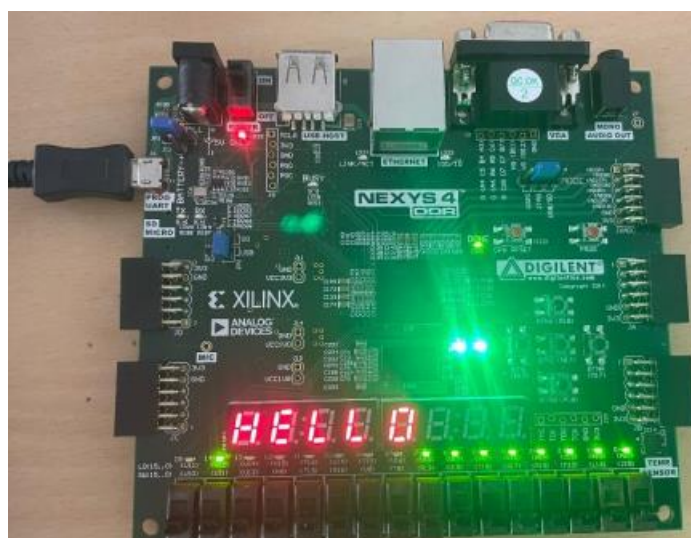
无操作 10s 倒计时结束，回到等待状态；



输入 0000，数码管依次显示



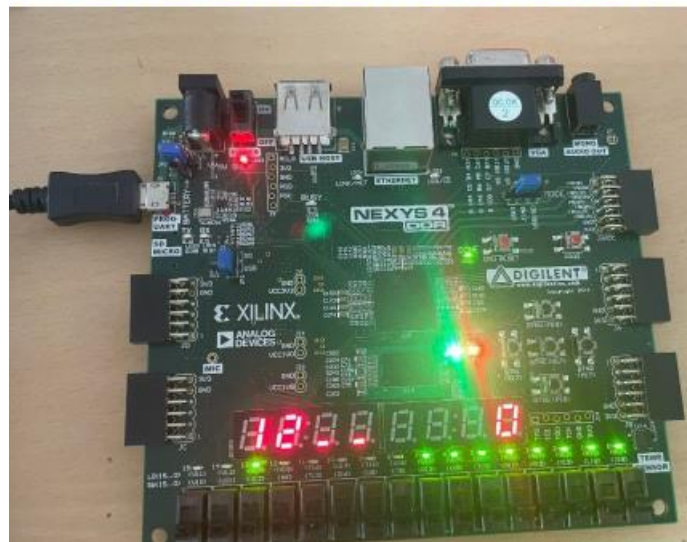
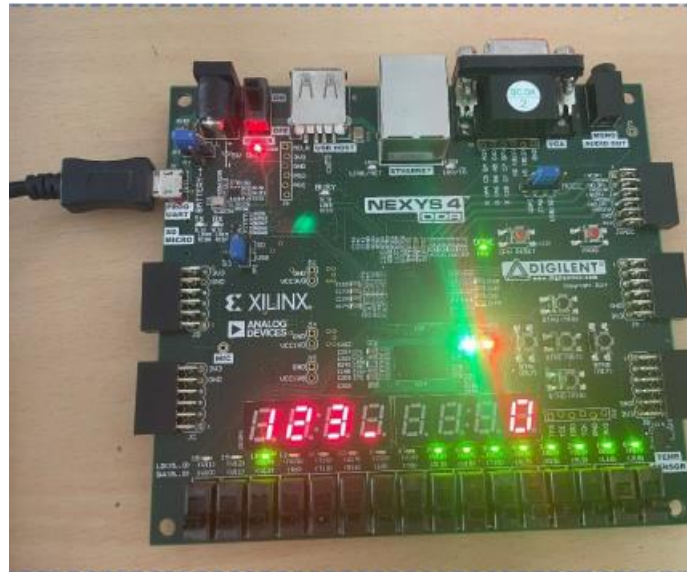
按下确认键，开锁，显示 HELLO，两个三色 LED 为绿色；



开锁状态下，20s 倒计时结束，回到等待状态；



按下退格键，输入由 123 退至 12；



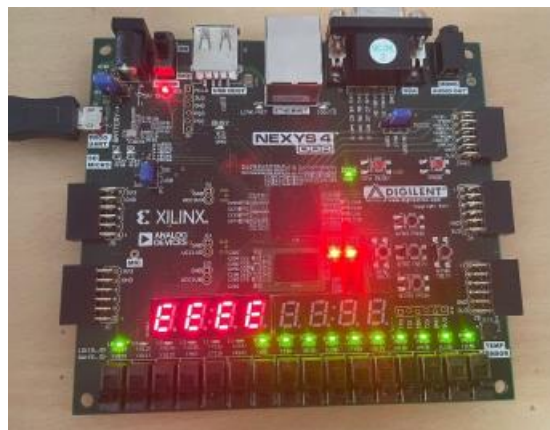
输入 1234，按下确认键，错误，显示 ERROR，两个三色 LED 显示黄色；

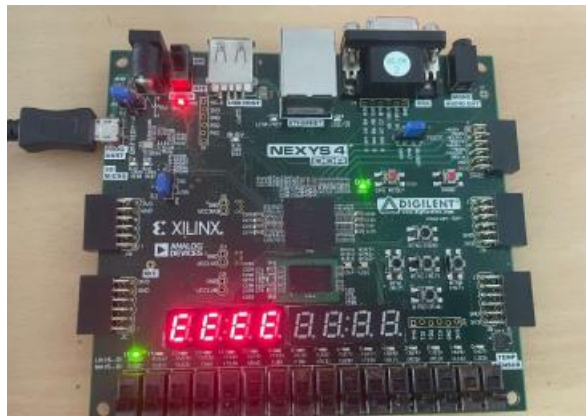


输入 1111，此时已错误一次，最右侧显示 1，确认，再次显示错误；

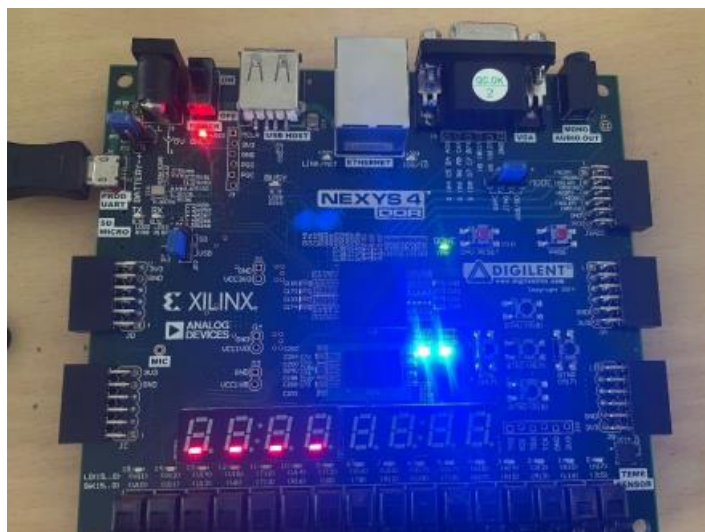


输入 8848，此时已错误两次，最右侧显示 2，确认，开始报警，LED 和三色 LED 闪烁；

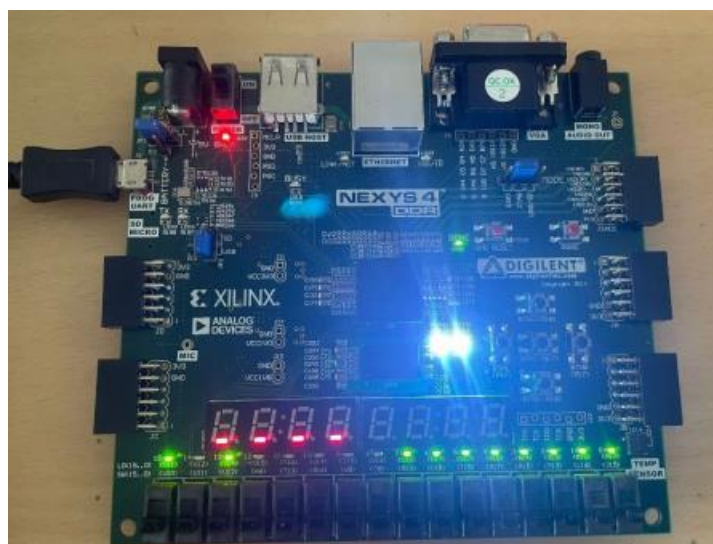




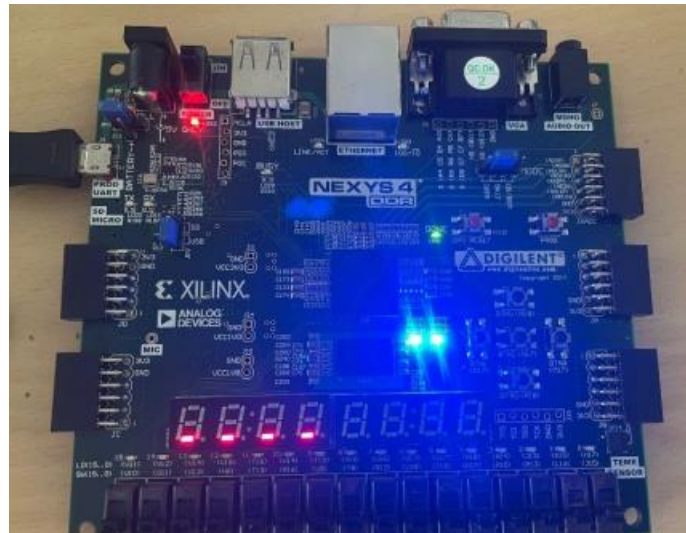
按下管理员按键，回到等待状态；



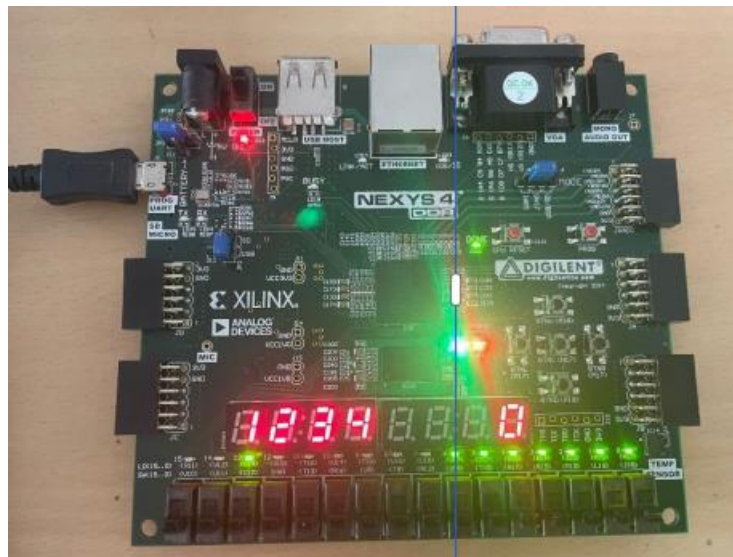
再次按下管理员按键，开始输入密码，我们输入 1234，两个三色 LED 显示白色；



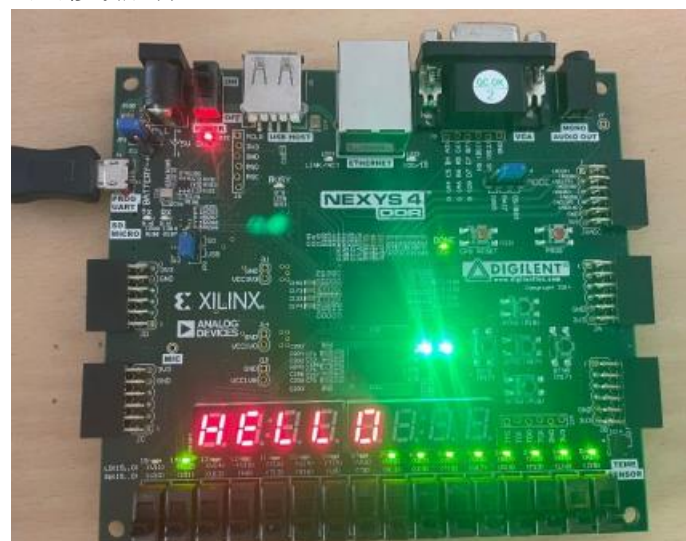
按下确认键，回到等待状态；



再次输入密码 1234;



按下确认键，开锁，密码修改成功



出现问题：多组模块整合在一起时，出现系统接口不一致和内部代码相互冲突问题，通过修改部分指令的运行逻辑和输入语言，达成系统运行稳定。

蓝牙连接系统需要转换相关键盘指令，加入新的源文件，修改相应接口。

系统优化问题：模块某一部分用不同代码都能实现运行，但是系统运行效率，速度却体现明显差异

实验结论：在数字系统实践课程设计中，我们利用 verilog 语言设计模仿模仿密码锁的工作过程，进行 RTL 级行为仿真完成密码锁的核心控制功能。并额外添加了蓝牙连接输入设置。

。不同条件，环境下影响密码锁的最终运行和显示结果。

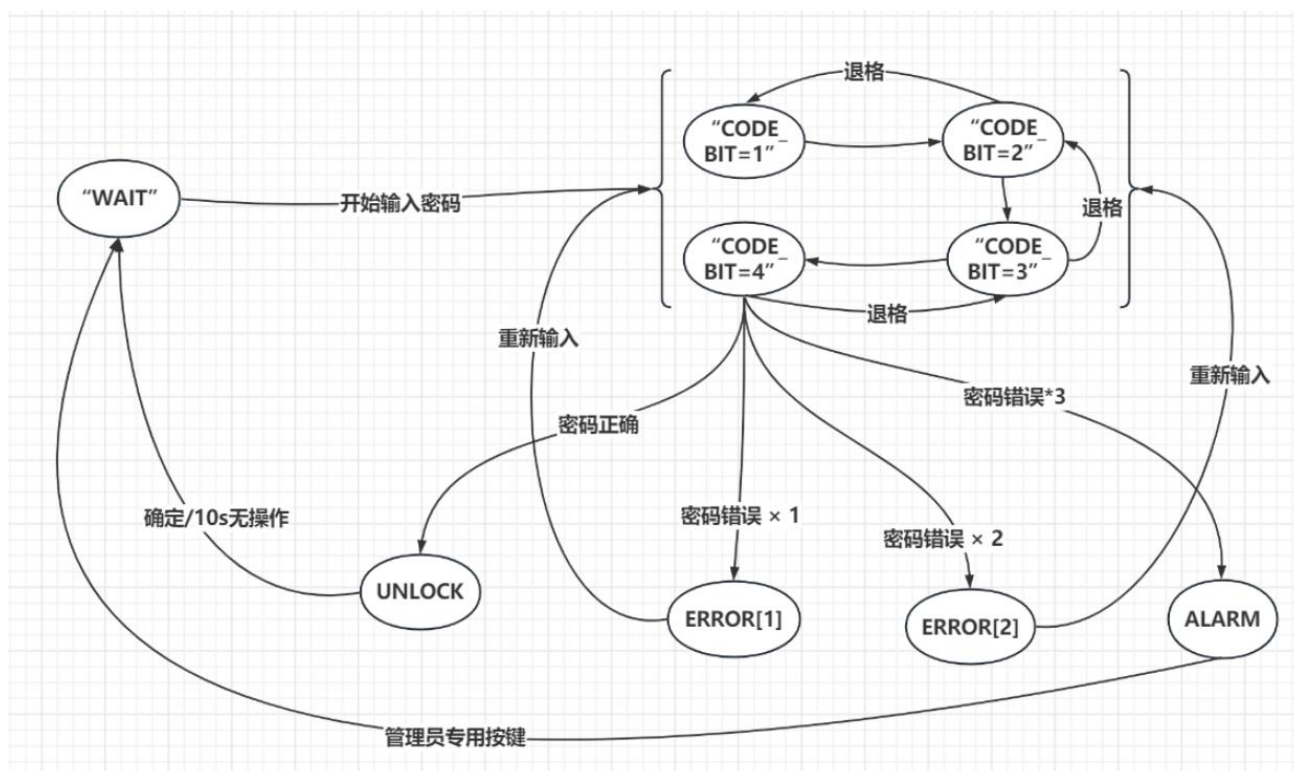
四. 设计特色，及设计过程中遇到的问题描述和解决方法

利用 Verilog 语言设计密码锁，运用相关知识设计系统。

1. 04023301 何君个人总结

在设计的不同阶段、每个人各自负责的工作中，都遇到了不同方面的问题，这些问题都让我们相当困扰，但也正是这些问题，在项目实现的各个过程中不断地加深我们对于数字系统设计的理解、加深我们对于硬件编程语言的理解。

在设计初期，对于数字系统本身、对于 HDL 和 Vivado 的理解尚不深入，设计上虽然能抓住梗概，但并不能向下进一步细化。比如，在设计讨论时，我们用了一个比较直观的 FSM 状态转换图，以为这个图就足以实现我们的设计，粗浅到甚至全是自然语言描述，但到后面才发现，真正困难的是信号的组织以及如何将自然语言转换成实际代码中的变量、分支和判断条件。



对于 HDL 本身，理解上也存在一些偏差，最典型的例子就是 HDL 中数的表示方法， i.e.,

[位数][位权][大小]

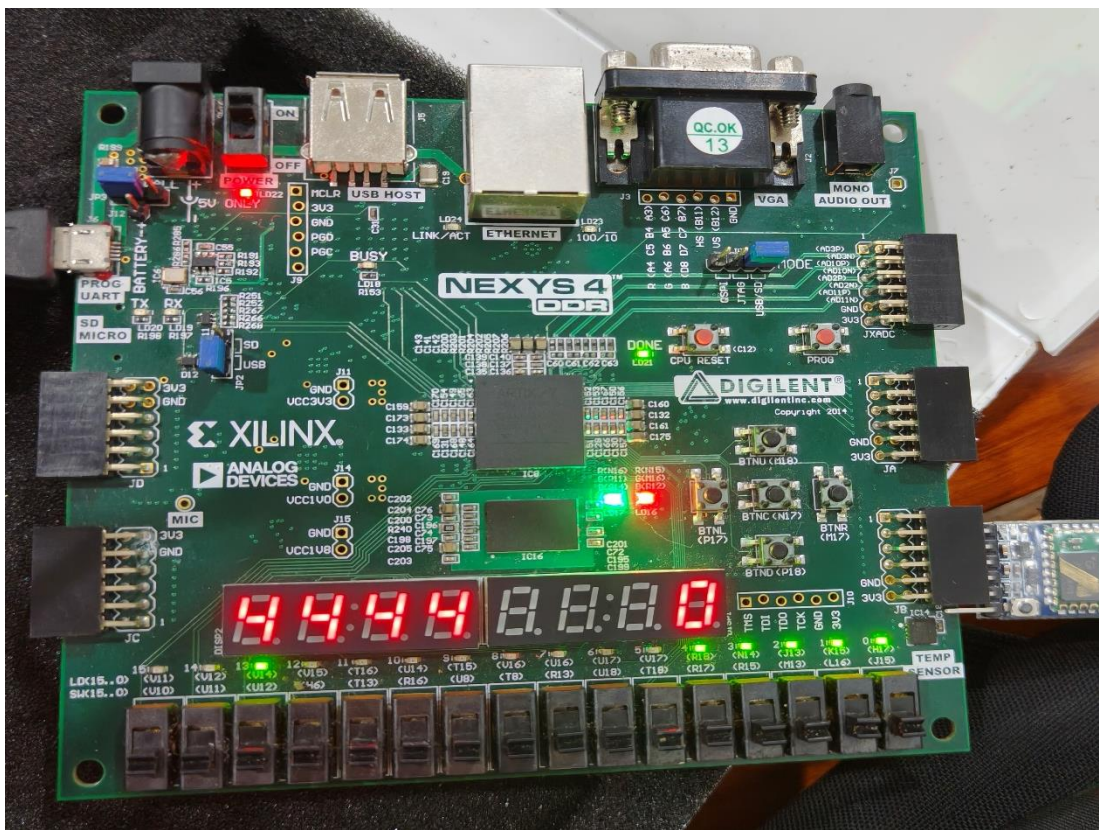
完全不同于以往的数制格式，我们曾经讨论很久，如 $4'hf$ ，到底应该理解成 $(000f)_{16}$ 还是 $(1111)_2$ ，后来突然醒悟，要从硬件电路的角度理解，在硬件电路中，怎么可能会有一位连线，上面的电平有 16 个分立取值，也就是说这种数的表示方法，实际上是对应着电路中总线里每根线的电平情况，每根线只有二值电平，所以要把后面数的大小，对应到二进制之后，才能确定前面总线的位数，至于造成混淆的 [位权]，只是为了人去读代码时更容易阅读数的大小，真正执行时还要变成二进制的总线。

另一个问题是 for 循环，到底有没有时序上的意义，还是仅复制将相同的模块并组织成并行电路？之后经过查找各种资料，我们认为应该按照后者理解。因为在电路中，在一个模块阻塞直到循环变量条件不满足，本身就会造成时序上的问题。

还有模块描述、模块调用时的端口类型规则，为什么在定义模块时必须要求 input 端口是一个 wire 类型变量，而上层模块实例化调用下层模块时就必须用一个 wire 类型变量作为 output 的承接，反而对于接入给 input 端口的信号类型不做要求？这实际上是从不同层次看模块的问题，从模块定义（模块内部）看，外部是一个黑箱，自己只能接收外部传来的信号，所以自己无法指定 input 的信号是什么类型，而只能用 wire 直连进行接收，但对于 output，自己当然可以决定如何产生这个信号，用 reg 驱动，或者是 wire 直连；从模块调用（模块外部）看，分配给下层模块的信号是自己可以决定的，但是下层模块对于自己又成了黑箱，下层模块处理完的信号，自己只能用 wire 直连线接收而不能指定类型，从更深层更硬件的角度理解，实际上是谁有对于信号的驱动权的问题，有驱动权就可以指定 reg，用寄存器驱动信号电平，否则只能用连线连同。

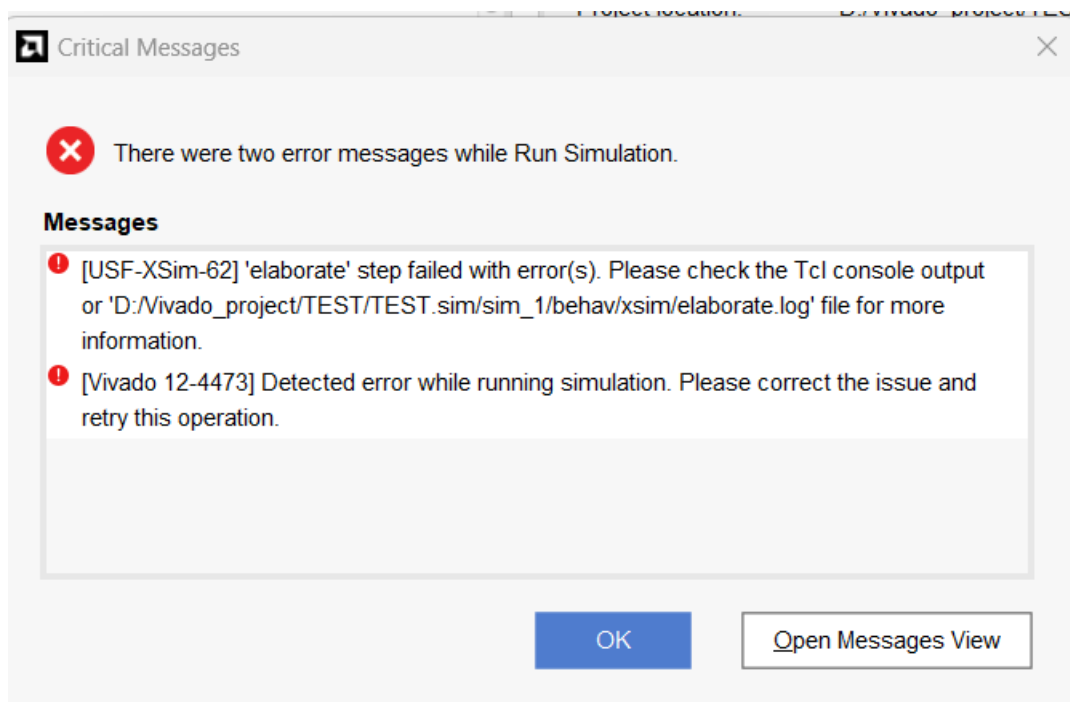
这些问题，让我们更清楚地理解了 HDL 与实际电路的映射关系，在代码风格上，我们的体会是，always 块是主导，而 assign 只是起到辅助作用。这是因为代码中 always、assign、module 的实例化全是并行执行的，那么 always @ (posedge CLK)，和 assign wire =，这两个并行到底是什么意思？前者时序相当清楚，而后者根本没有时序的概念，这两个并行，在时序上好像不可理喻。所以，always @ (posedge CLK) 真正控制系统的运行的节拍，对于一个 assign 来说，右侧变量一般是受节拍控制的，这种组合的含义就是，用 always（寄存器）进行驱动，并把信号进行连接，因为驱动源受到节制，线网的连接自然不会干扰时序，而 always @ (某个信号) 和 assign 的作用类似，只不过是 assign 搭配 wire 赋值，而 always 里面只能使用 reg，i.e.，赋值的目标信号类型不同罢了。

此外，在配置蓝牙输入的时候对于波特率的设置情况不够理解，何时传入数据？何时扫描数据并转义信号，都是一些疑难问题。另外，在成功实现蓝牙的协议匹配后，由于扫描信号与按键接入信号的扫描周期不匹配，导致输入密码接收到的总是是同一个信号，如下图所示，最终找到原因是由于设置蓝牙信号时将其设置为一个时钟周期持续信号而非单个脉冲信号，将其改为脉冲信号后其正常工作。



最困难的是时序问题，一个信号的响应，到底在哪个周期执行，执行出的信号，又是否能在合适的周期被接受到？一个信号，设置成一个高电平的持续量，还是一个周期的脉冲量？最后也没有完全想清楚这些设计问题（在按原本设计思路尝试实现时遇到，没有成功解决）

2. 04023426 唐熙文个人总结:



仿真环节出现部分问题，对系统代码的输入输出部分进行检查后，修改了程序问题，得以正常运行

多组模块整合在一起时，出现系统接口不一致和内部代码相互冲突问题，通过修改部分指令的运行逻辑和输入语言，达成系统运行稳定。

蓝牙连接系统需要转换相关键盘指令，加入新的源文件，修改相应接口。

系统优化问题：模块某一部分用不同代码都能实现运行，但是系统运行效率，速度却体现明显差异。不同条件，环境下影响密码锁的最终运行和显示结果。

五. 人员任务分配情况

04023301	何君	负责 TOP 顶层模块设计以及按键消抖功能实现，蓝牙模块无线传输功能以及蜂鸣器模块设计实现，以及工作报告的撰写	60%
04023426	唐熙文	工作报告撰写	40%

六. 其他

(根据需要补充或不填写)

成绩评定附件（以下仅由任课老师填写）：

	系统设计 (25+25)	仿真 (10+10)	系统演示 代码讲解 (20+20)	问题及回答 (15+15)	实验报告 (20+20)	分配比 例	平时分 (10+10)	总评分 数
小组						100%	-----	-----
学生：								
学生：								